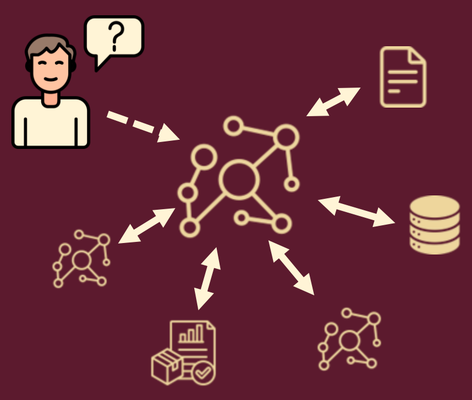
*Building a **Virtual Knowledge Graph** for **Data Lakes** Using Croissant Metadata*



Zoé Chevallier, Bianca Benelli, Matteo Lissandrini

{firstname.lastname}@univr.it

https://viasystemslab.github.io/

Enabling ***Queryable Data Lakes***

*A lightweight **Virtual Knowledge Graph** integration approach built on sources metadata*

Context and Challenges

- Data Lakes store ***heterogeneous data sources***: relational tables, semi-structured data, other tabular formats – *CSV files, Parquet, XML, JSON*
- Sources are stored in their ***own format*** and do not need to adhere to a specific schema
- Enabling ***unified and joint querying*** of heterogeneous data sources

➡ ***No obvious common query language that allows cross-source querying and semantic interoperability***

Key Concepts

Croissant Document: Metadata schema and vocabulary extension for machine learning datasets

- Capture the ***structural*** and ***semantic information*** beyond the schema and content of a dataset
- Use by ***machine learning frameworks*** to directly ingest the dataset with minimal code
- ***Croissant Documentation***: https://docs.mlcommons.org/croissant/

Virtual Knowledge Graphs: Implement OBDA (Ontology-Based Data Access) paradigm

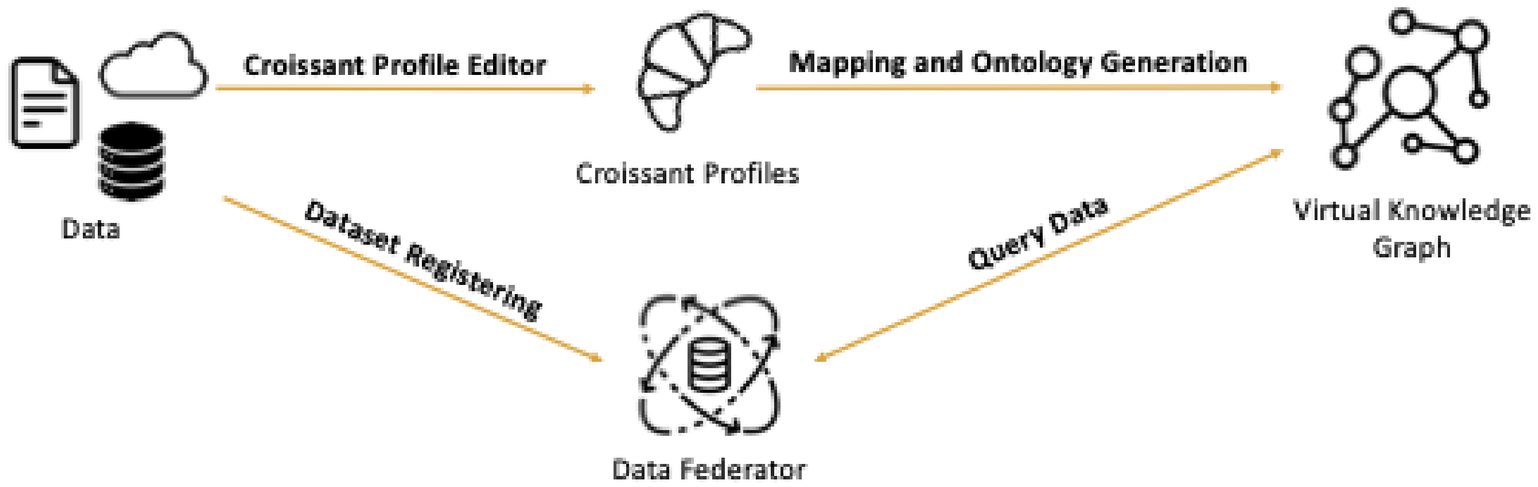
- Exposes ***heterogeneous*** data sources as ***virtual RDF graphs*** (without materialization)
- ***Unified query interface*** (SPARQL query language)
- Require ***ontology*** and ***mappings*** as input

Use Case

DataGEMS is a ***data discovery platform***, with a continuous ingestion of heterogeneous datasets from diverse sources and make them discoverable and ***queryable***

Meteorological data use case: among the different use cases supported by DataGEMS, the meteorological use case provides heterogeneous datasets from different sources and formats, including:
- Station metadata *(CSV file)*
- Historical observations *(PostgreSQL database)*

The integration allows users to ***query historical observations together with*** corresponding station information



Proposed Integration Pipeline of Data Lake Sources into a Virtual Knowledge Graph

Automatic Generation of Input Files

```
{ "@id": "era5_land",
  "name": "ERA5-Land Point-Based Weather Dataset",
  "description": "Point-based ERA5-Land dataset including temperature, precipitation values.",
  "@type": "sc:Dataset",
  ”recordSet”
      { "@id": "weather_variable",
        "@type": "cr:RecordSet”,
        "name": "weather_variable”,
        "key": { "@id": "id" },
        "field": [
            { "@type": "cr:Field",
              "@id": "id",
              "dataType":"xsd:integer”
            },
            { "@type": "cr:Field",
              "@id": "description",
              "dataType": "xsd:string”
            }
            ]},
            […]
```

Croissant

```
ex:era5_land/weather_variable a owl:Class;
prov:wasDerivedFrom ex:weather_variable_recordset.

ex:era5_land/weather_variable/id a owl:DataProperty ;
    rdfs:domain ex:era5_land/weather_variable ;
    rdfs:range xsd:integer ;
    prov:wasDerivedFrom ex:id .

ex:era5_land/weather_variable/description a owl:DataProperty ;
    rdfs:domain ex:era5_land/weather_variable ;
    rdfs:range xsd:string ;
    prov:wasDerivedFrom ex:description .
```

Ontology

```
<#TripleMapping_era5_land_1> a rr:TriplesMap ;
    rr:logicalTable [ a rr:LogicalTable ;
            rr:sqlQuery "SELECT * FROM lenses.era5_land.weather_variable" ] ;
    rr:predicateObjectMap [ a rr:PredicateObjectMap ;
            rr:objectMap [ a rr:ObjectMap ;
                    rr:column ”id" ;
                    rr:datatype xsd:integer ] ;
            rr:predicate <http://example.com/era5_land/weather_variable#id> ],

        [ a rr:PredicateObjectMap ;
            rr:objectMap [ a rr:ObjectMap ;
                    rr:column ”description" ;
                    rr:datatype xsd:string ] ;
            rr:predicate <http://example.com/
era5_land/weather_variable#description> ];
    rr:subjectMap [ a rr:SubjectMap ;
            rr:template “http://example.com/ era5_land/weather_variable/{id}”.
```

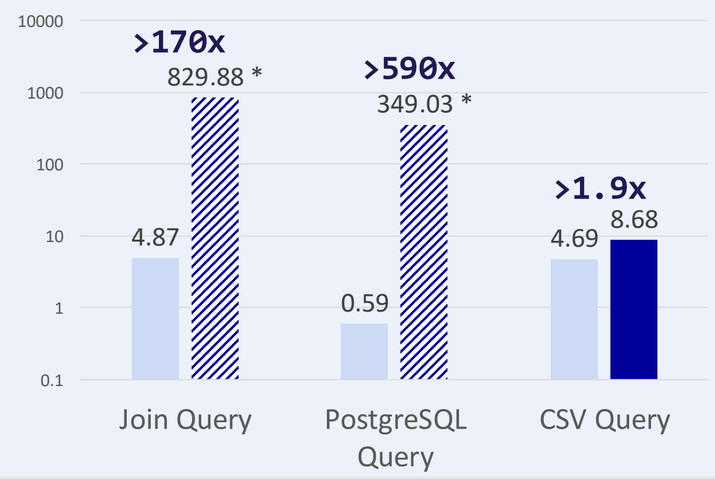
Mapping

```
{
  "relations": [
    {
      "name": ["lenses", ”era5_land”, ”weather_variable”],
      "baseRelation": [”era5_land", ”weather_variable"],
      "uniqueConstraints": {
        "added": [
          { "name": ”era5_land_weather_variable", "determinants": [”id"] }
        ]
      },
      "type": "BasicLens"
    }
]}
```

Lenses

Discussion and Preliminary Results

Preliminary results on the DataGEMS use case, comparing the running time with and without a lenses file (where the primary keys are declared)



Identified bottlenecks
- Need of having ***primary keys*** for each data source
- Use of ***performant formats***, e.g. convert CSV files to Parquet

Next Steps

1. *Extension to additional data types (e.g., JSON)*
2. *Automatic identification of primary keys*
3. *Incremental ontology alignment and mapping*

Contact us!

Your feedback is Welcome

github.com/VIASystemsLab/DataLake-VKG

This work has been supported by DataGEMS, funded by the European Union's Horizon Europe Research and Innovation program, under grant agreement No 101188416

DataGEMS

Funded by the European Union

DOI [TO UPDATE with QR CODE]